

홈 네트워크

HOME NETWORK

목차

01 홈 네트워크란?

02 사용된 패턴들

03 코드 실행 예시

04 마무리

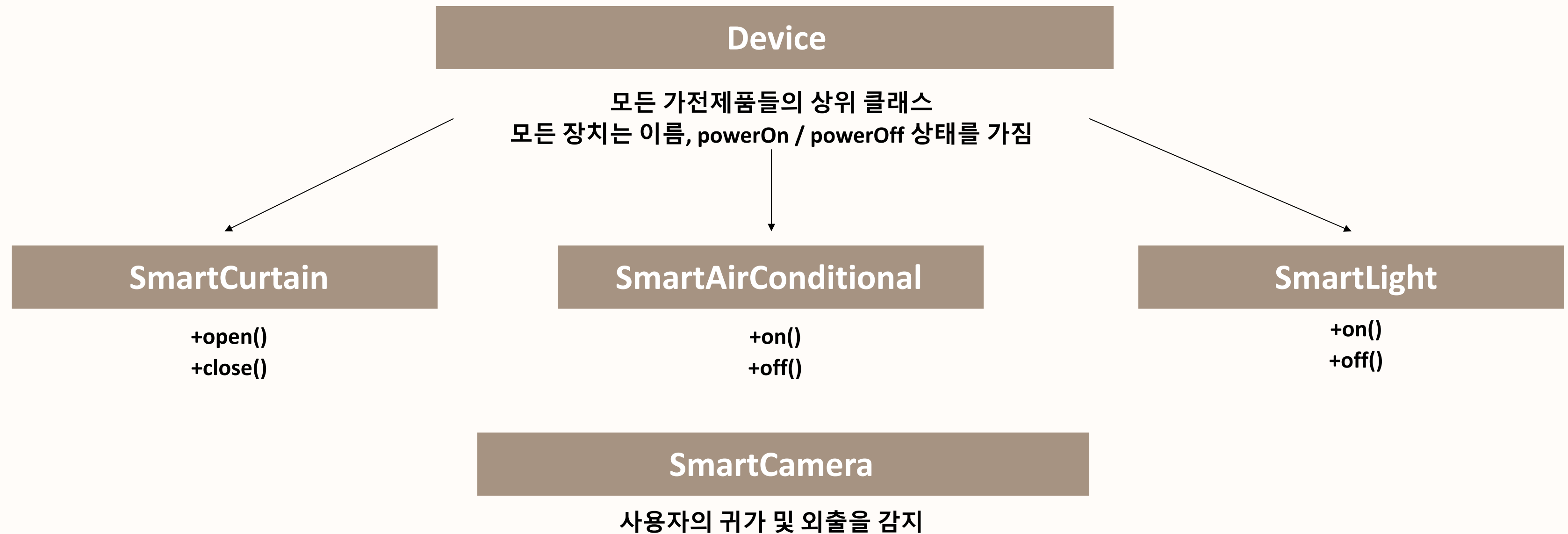
01 홈 네트워크

집 안의 여러 가전제품들을 연결하여 하나의 기기(ex. 스마트폰)로 편리하게 사용할 수 있게 하는 시스템

조명, 커튼, 에어컨 등 다양한 장치를 서로 연결

- 버튼을 사용하여 간단하게 장치를 제어
- 여러가지 명령들을 하나의 명령으로 묶어 간단하게 실행
- 복잡한 구조 → 패턴을 활용해 관리

Class 및 주요 장치



02 사용된 패턴들

Command pattern

요청을 객체 속에 캡슐화 해주는 패턴

Composite pattern

여러 개의 객체들로 구성된 복합 객체와 단일 객체를
클라이언트에서 구별없이 다루게 해주는 패턴

Observer pattern

한 객체의 상태 변화에 따라 다른 객체의 상태도 연동되도록
일대다 객체 의존 관계를 구성해주는 패턴

Facade pattern

복잡한 내부 시스템을 감추고 클라이언트가 쉽게
접근할 수 있는 인터페이스를 제공하는 패턴

Comand pattern

각 장치의 상태 변화 (powerOn / powerOff)를 간단하게 실행할 수 있도록 함

추상 클래스

```
/* ===== 2. Command 인터페이스 ===== */  
class Command {  
    execute(){ throw new Error('execute 필요'); }  
}
```

Comand를 상속하는 장치는 excute() 구현을 통해 장치의 상태를 제어

구현 예시

```
class LightOnCommand extends Command {  
    constructor(l){ super(); this.l = l; }  
    execute(){ this.l.on(); }  
}
```

SmartLight 클래스의 on() 메소드를 LightOnCommand로 캡슐화

Composite pattern

구현 예시

```
/* ===== 4. Composite ===== */  
class MacroCommand extends Command {  
  constructor(...cmds){ super(); this.cmds = cmds; }  
  execute(){ this.cmds.forEach(c => c.execute()); }  
}
```

```
const onLight = new LightOnCommand(light);  
const openCurt = new CurtainOpenCommand(curtain);  
const offAc = new AirconOffCommand(ac);
```

```
const morningMacro = new MacroCommand(onLight, openCurt, offAc);
```

여러 명령을 하나로 묶어서 순차적으로 처리

Constructor(...cmds): 여러 명령들을 묶는 배열을 생성
각각의 명령들을 순서대로 실행

각각의 명령들

Const morningMacro...: 세 명령이 순차적으로 실행

‘moningMacro’의 각 명령들인
LightOnCommand, CurtainOpenCommand, AirconOffCommand는
모두 Command 추상 타입을 상속하였으며, ‘moningMacro’는 각 Command를
컴пози션 함으로써, Macro를 실행할 명령들을 장착한다.

Observer pattern

```
const cam = new SmartCamera('현관 카메라');
cam.subscribe(new MacroRunner('arrival', comingMacro));
```

```
/* ===== 6-A. 카메라 Subject ===== */
class SmartCamera {
  constructor(name){ this.name = name; this.observers = []; }
  subscribe(o){ this.observers.push(o); }
  notify(e){ this.observers.forEach(o=>o.update(e)); }

  detectArrival(){
    console.log(this.name+' : 귀가 감지');
    this.notify({ type:'arrival' });
  }
}
```

```
/* ===== 6-B. Observer ===== */
class MacroRunner {
  constructor(evtType, macro){
    this.evtType = evtType; this.macro = macro;
  }
  update(e){
    if(e.type === this.evtType){
      console.log('자동화 : '+e.type+' → 매크로 실행');
      this.macro.execute();
    }
  }
}
```

‘cam’이라는 SmartCamera 객체 생성, 옵저버로
evtType : “arrival”, macro : ‘comingMacro’ 등록

cam.detectArrival()
귀가를 감지했을 경우 --> notify({ type: "arrival" })
notify()를 호출, 모든 Observer에게 ‘arrival’ 이벤트 전달

evtType === “arrival”인 Observer만 반응 --> comingMacro
comingMacro 실행

Facade pattern

```
/* ===== 7. Facade ===== */
class HomeFacade {
  static build() {
    const light = new SmartLight('거실 조명');
    const curtain = new SmartCurtain('거실 커튼');
    const ac = new SmartAirConditioner('거실 에어컨');

    const onLight = new LightOnCommand(light);
    const openCurt = new CurtainOpenCommand(curtain);
    const offAc = new AirconOffCommand(ac);
    const offLight = new LightOffCommand(light);
    const closeCurt = new CurtainCloseCommand(curtain);
    const onAc = new AirconOnCommand(ac);

    const morningMacro = new MacroCommand(onLight, openCurt, offAc);
    const sleepMacro = new MacroCommand(offLight, closeCurt, onAc);
    const leavingMacro = new MacroCommand(offLight, closeCurt, offAc);
    const comingMacro = new MacroCommand(onLight, openCurt, onAc);

    const cam = new SmartCamera('현관 카메라');
    cam.subscribe(new MacroRunner('arrival', comingMacro));
    cam.subscribe(new MacroRunner('departure', leavingMacro));

    return { morningBtn, sleepBtn, leavingBtn, comingBtn, cam };
  }
}
```

실제로 존재하는 객체로 생성

각 장치에 대한 Command 객체 생성
Command 패턴 구조로 되어있어
.execute()로 한 번에 실행

여러 개의 명령들을 한 번에 실행할 수
있게 해주는 객체 생성 (Composite pattern)

카메라(cam) 객체 생성
자동화 이벤트 등록(Observer pattern)

HomeFacade 클래스는 여러 장치, 명령, 이벤트 감지 등
복잡한 시스템을 단 하나의 build() 메소드 호출만으로 구성

03 코드 실행 예시

```
[Morning] 버튼  
거실 조명 : 조명 ON  
거실 커튼 : 커튼 OPEN  
거실 에어컨 : 에어컨 OFF  
  
현관 카메라 : 귀가 감지  
자동화 : arrival → 매크로 실행  
거실 조명 : 조명 ON  
거실 커튼 : 커튼 OPEN  
거실 에어컨 : 에어컨 ON
```

```
[Sleep] 버튼  
거실 조명 : 조명 OFF  
거실 커튼 : 커튼 CLOSE  
거실 에어컨 : 에어컨 ON  
  
현관 카메라 : 외출 감지  
자동화 : departure → 매크로 실행  
거실 조명 : 조명 OFF  
거실 커튼 : 커튼 CLOSE  
거실 에어컨 : 에어컨 OFF
```

Thank you

감사합니다.